

PROJECT PROPOSAL

ToQa — A Single-Store E-Commerce Platform for Modest Clothing

Node.js Bootcamp — Final Project Proposal

Submitted for Trainer Approval

August 2026

1. Project Title

Working title: ToQa

ToQa is retained as the default project name because it is short, ownable as a brand, and does not conflict with any well-known existing product. It is suitable for an e-commerce storefront and can carry a simple, memorable domain name.

Alternative Name Options (optional, for team consideration)

- Sitr (ستر) — Arabic for "modest covering"; communicates the modest-fashion niche directly and works well for a MENA-focused audience.
- Haya Wear — "Haya" evokes modesty/grace; reads clearly as a clothing brand in English-language markets.
- Deema Boutique — a warm, boutique-style name that signals a curated small-business storefront rather than a mass marketplace.

These are optional suggestions only. The team may keep ToQa as the final name; no concept change is implied by any naming choice.

2. Team Members

The project is developed by a team of six trainees, as approved by the trainer:

#	Team Member
1	Abdulrahman Nabil
2	Emad Memi
3	Mohmed Abbas
4	Ahmed Ashraf
5	Kerelos Reda
6	Ahmed Bakr

Team size and composition are fixed as approved. Role distribution across backend modules (e.g., catalog/inventory, cart/checkout, orders/coupons, auth/admin, reviews/wishlist, QA/documentation) will be defined internally by the team and is not part of this proposal.

3. Project Overview

ToQa is a single-store e-commerce backend for a modest-clothing retailer. It allows customers — with or without an account — to browse products, choose a specific size/color variant, manage a cart, check out, choose between two supported payment methods, and track their order. A protected admin area lets store staff manage products, variants, stock, orders, coupons, customers, and reviews.

The project is scoped as a single store, not a multi-vendor marketplace, and is built entirely with the Node.js Bootcamp's required technology stack: Node.js, Express.js, MongoDB, and Mongoose, with JWT-based authentication and role-based authorization layered on top.

4. Problem Statement

Small and medium modest-clothing businesses commonly run sales through informal channels (social media direct messages, manual order books) that do not scale and are error-prone. This creates several concrete problems:

- No structured way to track which product variant (size/color) is actually in stock, leading to orders for unavailable items.
- No centralized order record, making it hard to track status, delivery information, or payment status.
- No easy way to run discounts or promotions in a controlled, auditable way.
- Customers are often forced to message the seller just to check availability, which is slow and does not scale.
- No self-service order history for repeat customers, and no structured collection of reviews to build trust.

ToQa addresses these problems directly with a variant-aware inventory model, an enforced stock-validation flow, structured order and coupon management, and optional customer accounts.

5. Proposed Solution

ToQa is a REST API backend, built with Express and MongoDB/Mongoose, that models the store's real business entities — products, variants, stock, carts, orders, coupons, reviews, and users — and enforces the business rules described in this proposal at the API layer, not only in the client. Guest checkout removes friction for casual buyers, while optional accounts add value (order history, wishlist) for returning customers. A JWT-protected, role-based admin area gives the business owner full operational control without exposing that control to customers.

6. Project Objectives

- Deliver a functioning, variant-aware product catalog with accurate per-variant stock.
- Support guest and authenticated shopping flows without forcing registration.
- Enforce inventory rules on the backend so stock cannot be oversold.
- Provide a secure, role-based admin system for day-to-day store operations.
- Support two realistic, non-gateway payment methods (Cash on Delivery, Electronic Wallet).
- Implement coupons, reviews, wishlist, search, filtering, and pagination as described.
- Produce clean, understandable, well-documented code that every team member can explain and defend in the technical discussion.
- Meet every Node.js Bootcamp technical and process requirement (Git history, README, Postman, .env.example, etc.).

7. Target Users

- Guest shoppers who want to browse and buy without creating an account.
- Registered customers who want order history, wishlist, and a saved profile.
- The store administrator/owner who manages catalog, inventory, orders, and coupons.
- Small-to-medium modest-clothing businesses as the underlying business persona this system is built for.

8. User Roles

Role	Authentication	Capabilities
Guest	None	Browse, search, filter, cart, guest checkout, place orders
Customer	JWT (optional)	All guest capabilities plus profile, order history, wishlist, reviews
Admin	JWT (required) + role check	Full management of products, variants, inventory, orders, coupons, users, reviews, dashboard

9. Guest User Features

- Browse, search, and filter products
- View product details, including per-variant availability
- Select size and color
- Add/update/remove cart items
- Checkout with delivery details and a chosen payment method
- Place an order without creating an account

10. Registered Customer Features

- Register, log in, log out (JWT-based session)
- Manage profile information
- View order history and order details
- Use a wishlist
- Submit reviews, preferably restricted to purchased products

Registration remains optional; guests are never blocked from purchasing.

11. Admin Features

- Secure login via JWT with role-based authorization
- Manage products, variants, categories, sizes, and colors

- Manage inventory, including low-stock visibility
- Manage orders and order status
- Manage coupons
- Manage customers and reviews
- View basic dashboard metrics: order count, product count, low-stock items, recent orders, basic sales figures

12. Product Management

Admins can create, update, deactivate, and delete products, and organize them by category. Each product acts as a container for one or more variants (see Section 13). Product-level fields include name, description, category, base images, and status; variant-level fields carry size, color, price adjustments (if any), and stock.

13. Product Variants

Every product is sold through variants defined by size and color, each carrying its own stock count (for example, Black/S = 10, Black/M = 5, Black/L = 0, Black/XL = 7). A different color may expose a different set of available sizes. This variant model is the backbone of the inventory system and is enforced consistently across catalog display, cart validation, and order creation.

14. Inventory and Out-of-Stock Management

Stock is tracked per variant, not per product. A variant with zero stock stays visible in the UI but is marked unavailable and cannot be selected or added to the cart; if every variant of a product is unavailable, the product is shown as out of stock. Critically, this rule is enforced on the backend at cart-update and order-creation time, not only in the frontend, so a stale client cannot bypass it. Admins can view low-stock variants to plan restocking.

15. Shopping Cart

Both guests and authenticated customers get a cart that supports adding a specific product variant, changing quantity, removing items, and applying a coupon. The backend recalculates subtotal, discount, and final total server-side, and re-validates that each variant is still available and in sufficient stock before checkout — the cart is never trusted to carry authoritative pricing or availability.

16. Checkout

Checkout collects customer name, phone number, governorate, address, delivery notes, chosen payment method, and optional order notes. Guests complete checkout with this information alone; logged-in customers have the order linked to their account automatically. Checkout is the point where stock and coupon validity are re-checked immediately before the order is created.

17. Payment Methods

Two payment methods are supported, matching the Bootcamp's scope constraints: Cash on Delivery, and Electronic Wallet, represented as a simple business flow in which the customer selects wallet payment and follows store-provided instructions rather than integrating a real payment gateway. Each order stores the chosen payment method, a payment status, and any relevant payment reference information. No real card gateway (Stripe, PayPal, Visa/Mastercard) is included in this scope.

18. Order Management

Each order records the customer (guest details or linked account), ordered variants and quantities, unit prices at time of order, any discount applied, shipping/delivery information, payment method and status, order status, and creation date. Order status progresses through Pending, Confirmed, Processing, Shipped, Delivered, and Cancelled. Customers can view their own order history when logged in; admins can view and update any order.

19. Wishlist

Wishlist is available only to authenticated customers, since it depends on a persistent account. Guests are not offered a wishlist, consistent with the approved scope.

20. Reviews and Ratings

Authenticated customers can leave a rating and comment on a product, preferably restricted to customers who actually purchased that product, to keep reviews credible. Admins can moderate (view/remove) reviews when necessary.

21. Coupon System

Admins define coupons with a code, either a percentage or fixed discount, an expiration date, a usage limit, an optional minimum order amount, and an active/inactive flag. Customers may apply a valid coupon at checkout; the backend independently validates expiry, usage limit, and minimum order amount before applying any discount.

22. Search and Filtering

The product listing API supports search by name and filtering by category, price range, size, color, and availability, so customers can narrow results without relying on client-only filtering logic.

23. Pagination

Product listing endpoints are paginated on the backend (page/limit-style parameters), keeping response sizes predictable as the catalog grows.

24. File Upload

Admins can upload product images through a dedicated upload endpoint. Uploaded files are validated for type and size before being accepted, demonstrating secure file-handling practice rather than trusting client-supplied files blindly.

25. Authentication and Authorization

Authentication uses JWT for both customers and admins. Customers may optionally register and log in; admins must always authenticate. Role-based authorization middleware protects all administrative routes, and passwords are hashed (e.g., bcrypt) before storage. No JWT secret or credential is ever committed to source control.

26. Security

- Password hashing and JWT-based authentication
- Role-based authorization on protected/admin routes
- Server-side input validation on all write endpoints
- Secrets and configuration kept in environment variables, never committed
- Centralized error handling with consistent, non-leaky error responses
- Secure file-upload validation (type/size checks)
- Rate limiting on sensitive endpoints (e.g., auth) where appropriate
- Correct, consistent HTTP status codes

27. Validation and Error Handling

Every write endpoint validates its input server-side (registration, login, product/variant creation and update, cart operations, checkout, coupons, reviews, file uploads). A centralized error-handling middleware returns consistent JSON error responses for validation errors, authentication/authorization errors, not-found errors, duplicate data, invalid variant selection, insufficient stock, invalid coupons, invalid payment information, and unexpected server errors.

28. Logging

The system logs important events and errors — server errors, authentication events, key administrative actions, and order-related failures — using a lightweight logger (e.g., Winston or Morgan combined with a structured logger). Passwords, JWT secrets, and other sensitive credentials are never logged.

29. Technology Stack

Required

- Node.js, Express.js

- MongoDB, Mongoose
- JWT Authentication, Role-Based Authorization
- Server-side validation (e.g., Joi/express-validator)
- Centralized error handling
- REST API design
- Environment variables (dotenv)
- Git and GitHub
- File upload handling (e.g., Multer), pagination, search, filtering, logging

Recommended, to be used where practical

- Docker (for consistent local setup)
- Unit testing (e.g., Jest)
- Swagger for live API documentation
- Postman Collection for manual/API testing

30. Backend Architecture

The backend follows a conventional layered structure to keep the codebase understandable to every team member and defensible in the technical discussion:

- routes/ — route definitions per resource
- controllers/ — request/response handling
- services/ — business logic (stock checks, coupon rules, order totals)
- models/ — Mongoose schemas
- middlewares/ — auth, role checks, error handling, upload handling
- validators/ — request validation schemas
- utils/ and config/ — shared helpers and environment/config setup

No additional architectural layers (e.g., CQRS, microservices) are introduced; the goal is maintainability and clarity, not architectural complexity.

31. REST API

The backend exposes a RESTful API organized around the platform's core resources. At a high level, the anticipated API groups are:

- Authentication (register, login, logout, token refresh if included)
- Users / Customer profile
- Products and Categories
- Product Variants (sizes, colors, stock)
- Cart
- Orders
- Coupons

- Reviews
- Wishlist
- Admin (dashboard and management endpoints)

Detailed endpoint lists, request/response contracts, and route-level specifications are intentionally out of scope for this proposal and will be produced during the SRS/technical design phase.

32. API Documentation

The project will ship a Postman Collection covering the main flows, and Swagger documentation describing available endpoints, authentication requirements, request bodies, and response shapes, satisfying the Bootcamp's documentation requirement.

33. Testing

Unit testing (recommended) will prioritize the logic most likely to break or most central to business correctness:

- Authentication and authorization checks
- Input validation rules
- Cart subtotal/discount/total calculations
- Coupon validation logic
- Stock validation on cart update and order creation
- Order creation flow

34. Deployment Considerations

The application will run from environment-variable-driven configuration so it can move between local, staging, and any future hosting environment without code changes. Docker is recommended, if it can be added without unnecessary complexity, to make local setup reproducible across all six team members' machines and to ease any future deployment. Formal production deployment (e.g., specific cloud provider setup) is not a required Bootcamp deliverable and is not detailed further here.

35. Project Workflow

Guest

Browse Products → View Product → Select Size/Color → Add to Cart → Checkout → Enter Delivery Information → Select Payment Method → Place Order

Registered Customer

Register/Login → Browse Products → Add to Cart → Wishlist (optional) → Checkout → Select Payment Method → Place Order → View Order → Receive Order → Submit Review

Admin

Admin Login → Dashboard → Manage Products → Manage Variants → Manage Inventory → Manage Orders → Manage Coupons → Manage Users → Manage Reviews

36. Project Scope

ToQa is deliberately scoped as a single-store e-commerce backend — not a marketplace, not a payment processor, and not an ERP. The features listed throughout this proposal represent a realistic, demonstrable scope for a six-person team within a Bootcamp timeline. Quality of implementation and depth of understanding take priority over feature count.

37. Features Excluded from Current Scope

- Multi-vendor marketplace functionality
- Real payment gateways (Stripe, PayPal, Visa/Mastercard)
- AI-based product recommendations
- Microservices architecture
- Real-time chat/support
- Complex shipping-carrier integrations
- Advanced business intelligence / analytics
- Full ERP-style functionality

Any of these could be introduced later only with explicit trainer approval and a clear business justification.

38. Future Enhancements

- Real payment gateway integration once the business is ready to accept card payments
- Email/SMS notifications for order status changes
- A lightweight recommendation feature based on purchase history
- An admin analytics dashboard beyond the current basic metrics
- Multi-language storefront support

39. Expected Project Value

ToQa gives a small modest-clothing business a real, structured path from informal social-media selling to an accurate, variant-aware online storefront: no more promising a size that is not actually in stock, a single source of truth for orders and payment status, self-service order tracking for repeat customers, and controlled discounting through coupons. This is a genuine, demonstrable business problem with a scoped, achievable solution — not a feature checklist assembled to look impressive.

40. Why This Project Is Suitable for the Node.js Bootcamp

The project exercises every core Bootcamp competency in a single, coherent domain: relational-style modeling in MongoDB/Mongoose (products, variants, orders), JWT authentication and role-based authorization, server-side validation and centralized error handling, REST API design, file upload, search/filter/pagination, and security fundamentals. It is complex enough to sustain a meaningful technical discussion (variant-level inventory, coupon validation, guest-vs-account checkout) while staying realistic for six trainees to build, understand, and defend within the Bootcamp timeline.

Scope Recommendations

A critical review of the agreed scope, rather than uncritical agreement, follows. No business decision listed above is contradicted; the notes below flag sequencing, risk, and effort trade-offs the team should weigh going into the SRS phase.

Essential (build first, core to the business problem)

- Product catalog with variant-level stock (Sections 12–14) — this is the project's central value proposition; everything else depends on it.
- Cart and checkout with server-side stock re-validation (15–16) — without this, the inventory model has no teeth.
- Order creation and status management (18)
- JWT auth, role-based authorization for admin (25)
- Centralized validation and error handling (27)
- Search, filtering, pagination (22–23) — small but high-signal Bootcamp requirements

Important but lower risk if trimmed later

- Coupons (21) — clear business value, but validation logic (expiry, usage limit, minimum order) adds real complexity; keep, but implement after the core order flow is solid.
- Wishlist (19) — low implementation risk and cheap to demonstrate; a reasonable, safe scope-filler once the core flow works.
- Reviews (20) — the "purchase-verified reviewer" rule is worth keeping because it is a good technical discussion point, but it does add a coupling between Orders and Reviews that should be modeled carefully.

Higher effort relative to value — worth watching

- Electronic Wallet as a "simple business flow" needs a precise definition early (what exactly does the customer submit, and what does "payment status" mean without a real gateway?), or it risks being vague at the technical discussion. Recommend the team write one paragraph defining this flow exactly before the SRS.
- Admin dashboard metrics (11) are easy to over-scope into a mini-BI tool. Recommend capping this strictly at the four listed metrics (order count, product count, low-stock items, recent orders) plus one basic sales figure — resist adding charts or trend analysis.
- File upload validation (24) is simple to implement but easy to under-secure (MIME-type spoofing, oversized files); this should not be treated as a minor feature during security review.

Potential conflicts / risks identified

- Stock must be re-checked at both cart-update time and order-creation time — a variant that was available when added to the cart may sell out before checkout completes. The proposal already requires backend enforcement (Section 14); the team should explicitly decide the re-check happens at checkout, not only at add-to-cart.
- Coupon minimum-order-amount and stock validation both run at checkout; the team should agree on validation order (stock first, then coupon, or vice versa) so error messages are predictable and testable.

- Guest orders and account linking: if a guest later registers with the same phone/email, the proposal does not require retroactively linking past guest orders — this is worth a one-line decision ("guest orders are not merged into a new account") to avoid ambiguity during development.
- Reviews restricted to "purchased product" requires checking Delivered/Confirmed order status, which slightly couples the Reviews module to Orders; keep this coupling minimal and one-directional.

Nothing recommended for removal

No feature in the current scope is unnecessary; the scope as approved is realistic for six trainees. The recommendation is sequencing and rigor (validate the core inventory/cart/order loop thoroughly before layering coupons, wishlist, and reviews on top), not cutting features.

Bootcamp Requirements Coverage

Recommended (non-mandatory) items are explicitly marked as such; all other rows are treated as mandatory Bootcamp requirements.

Bootcamp Requirement	How the Project Satisfies It
Node.js	Entire backend runtime is Node.js.
Express.js	All routing, middleware, and request handling built on Express.
MongoDB	Primary datastore for all business entities.
Mongoose	Schema definition, validation, and querying layer over MongoDB.
JWT	Used for both customer (optional) and admin (required) authentication.
Validation	Server-side validation on all write endpoints (auth, products, cart, checkout, orders, coupons, reviews, uploads).
Error Handling	Centralized error-handling middleware with consistent JSON error responses.
REST API	Resource-oriented REST endpoints grouped by domain (Section 31).
Environment Variables	.env for secrets/config, never committed; .env.example provided.
Git	Feature-based commits throughout development, made by all six members.
GitHub	Central shared repository, pushed to regularly.
README	Setup, environment, and usage instructions included in the final deliverable.
Postman	Postman Collection covering the main API flows.
Swagger (Recommended)	Swagger docs describing endpoints, auth, requests, and responses.
Role-Based Authorization	Middleware distinguishes customer vs admin access on protected routes.
File Upload	Product image upload with type/size validation (Multer or equivalent).
Pagination	Backend pagination on product listing endpoints.
Search	Backend search by product name.

Bootcamp Requirement	How the Project Satisfies It
Filtering	Backend filtering by category, price, size, color, and availability.
Logging	Server errors, auth events, and key admin actions logged; secrets never logged.
Docker (Recommended)	Used for reproducible local setup if it can be added without unnecessary complexity.
Unit Testing (Recommended)	Jest-based tests targeting stock validation, coupon logic, auth, and order creation.
Multiple Git Commits	Ongoing, meaningful commits from all six members across the build.
Real Git History	History preserved and never deleted before submission.
Security	Hashing, JWT, RBAC, validation, env vars, rate limiting, and safe file upload together (Section 26).
Understandable Code	Layered architecture (routes/controllers/services/models) kept intentionally simple.
Real Business Value	Solves a genuine small-business inventory/ordering problem (Sections 4 and 39).

Final Recommendation

This project is ready to be submitted to the trainer for approval. The scope is realistic for a six-person team, stays within a single-store e-commerce boundary rather than expanding into marketplace or ERP territory, and maps cleanly onto every mandatory Node.js Bootcamp requirement. The variant-level inventory model and the guest-vs-account checkout flow give the project enough real business logic to sustain a serious technical discussion, without introducing complexity (real payment gateways, microservices, multi-vendor support) that would be difficult for six trainees to fully own and explain.

Before moving into the SRS phase, the team should resolve the small number of open decisions flagged in "Scope Recommendations" above — most importantly, the exact definition of the Electronic Wallet flow, the order of stock-versus-coupon validation at checkout, and the decision not to merge guest orders into later accounts. None of these require changing the approved concept; they simply need to be written down before detailed design begins.